



Amarisoft
TechAcademy

Search

Internal Search
chatTechnote

Home

Email
About

Community

Band/BW Calc
3GPP Guru

L
LT

LTE	LTE Adv	5G/NR	6G	NTN	2G/3G	LTE - M1	NB-IoT	IoT	IMS/S
WiFi	SDR	Comm. Tech.	RF	AI/ML/LLM	Embedded Sys	IP/Network	Linux	MechEngineering	Open R
Eng Math	Differential Eq	Matlab	Python	C,C++	Arduino/Raspberry	Web Sim	NVIDIA	JavaScript	Electron

5G/NR

5G/NR

- Quick Reference
- System Architecture
- Flow across System

PHY and Protocol

- Frame Structure
- Numerology
- Waveform
- Frequency Band
- BWP
- Synchronization
- Beam Management
- CSI Framework
- Channel Mapping
- CORESET
- DCI
- SLIV
- UCI/PUCCH
- Reference Signal
- MAC
- RLC
- PDCP
- SS Block
- Scheduling
- MIB / SIB
- RACH
- RRC Overview
- RrcReconfiguration
- NSA/ENDC
- SA/Initial Attach
- UE Capability
- Measurement Rep.
- Paging
- Power Control
- MIMO Config. DL
- MIMO Config. UL

NAS / Core

- Registration
- Network Slice
- Core Architecture

Misc.

- Field Test
- OTA(Over The Air)
- Release 16
- Release 17
- Massive MIMO ?
- WhyMassiveMIMO ?
- Propagation Model
- 5G Success
- 5G Challenges
- Acronymss
- Qualcomm Video

=	-43%	-40%	-27%	-20%
	R\$ 90,06	R\$ 538,80	R\$ 217,54	R\$ 3

LDPC

LDPC stands for "Low-Density Parity-Check". It's a method used in communication systems for encoding and decoding data to detect and correct errors. LDPC codes are a type of linear error-correcting code that has found extensive application in modern communication technologies. LDPC codes are a powerful and efficient method for error correction in digital communication systems, enabling reliable transmission even in challenging and noisy environments.

Here are some key points about LDPC:

- Error Correction:** LDPC codes are particularly effective for correcting data transmission errors in noisy communication channels. They are used in scenarios where accurate data transmission is critical, such as in satellite communication, wireless networks, and data storage.
- Sparse Parity-Check Matrix:** The 'low-density' in LDPC refers to the parity-check matrix used in these codes, which has a relatively small number of non-zero elements. This sparsity is what makes LDPC codes computationally efficient for error correction.
- Decoding Algorithm:** LDPC codes are usually decoded using iterative probabilistic decoding algorithms, such as the belief propagation (BP) algorithm. These algorithms are efficient and can handle a large number of errors.
- Applications:** LDPC codes are widely used in modern communication standards, including Wi-Fi (as part of the IEEE 802.11 standard), 5G cellular networks, and digital television (DVB-T2 and DVB-S2).
- History and Development:** LDPC codes were first introduced by Robert G. Gallager in his doctoral dissertation at MIT in 1960. However, they were not widely used until the 1990s when advancements in digital technology made their implementation more feasible.
- Performance:** LDPC codes are known for their near-Shannon-limit performance. The Shannon limit is a theoretical maximum efficiency for data communication in a channel with a given level of noise.

This note is about LDPC with the perspective of the implementation in 5G/NR. Followings are the topics to be covered in this note

- [LDPC in NR](#)
- [What is QC-LDPC?](#)
- [What is Lifting ?](#)
- [Why LDPC is Compute-Intensive ?](#)

NOTE : However in detail I try to describe the process, it would not be enough to understand every single step of this process unless you really try things on yourself. But implementing these kind of things on our own is a huge challenge. As a stepping stone between high-level conceptual understanding and real implementation, I wrote a simulation-based tutorial for this process with the help of AI (Gemini and ChatGPT). This tutorial would show you step by step process from Transport Block Size to a complete H matrix, even complete system equations. Check out [this note](#).

LDPC in NR

NR selected LDPC for the data channels (PDSCH and PUSCH) due to its excellent performance and suitability for high-throughput, parallel processing.

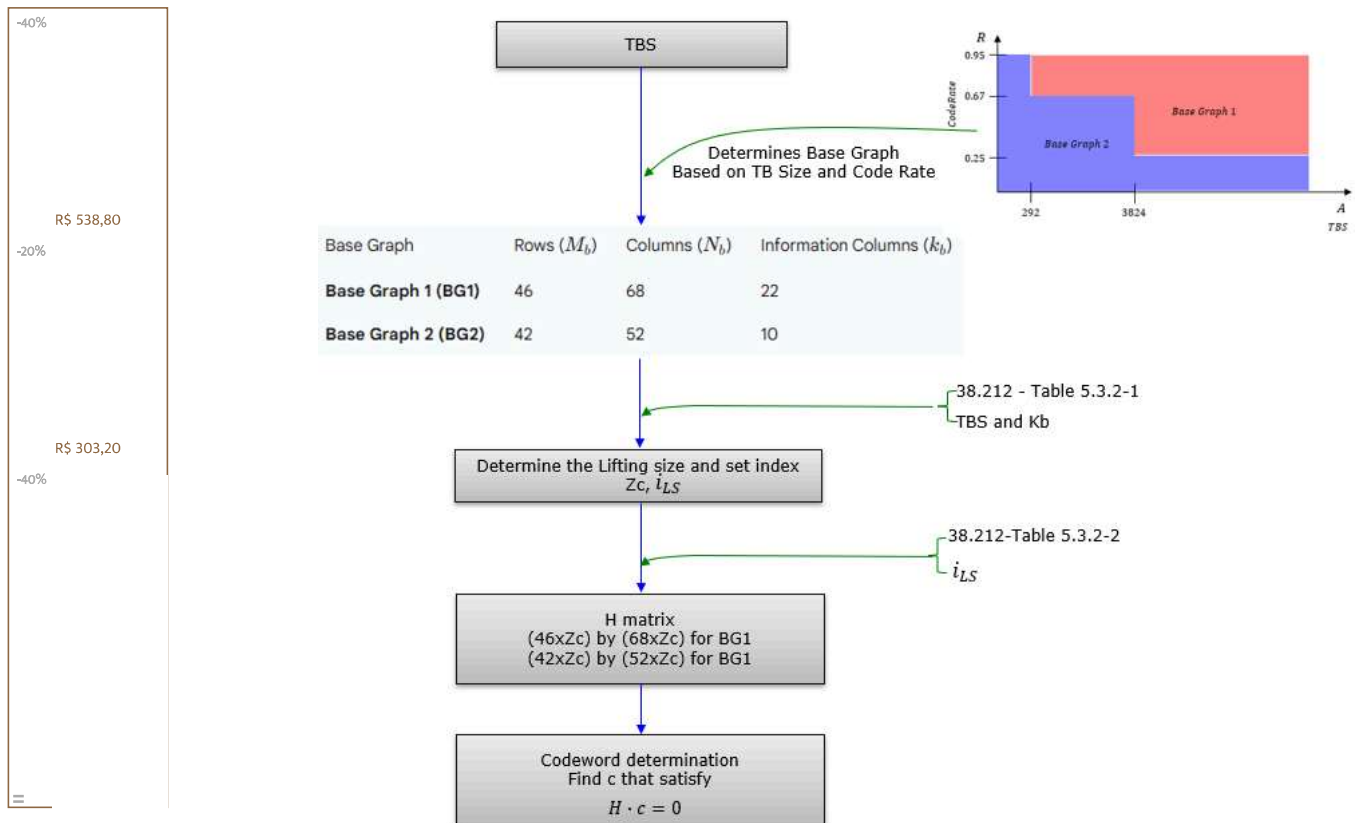
High Level Procedure of LDPC Encoding

This is about LDPC encoding process at the very high level based on my own understanding.

This procedure shows that NR LDPC encoding is not a single fixed code. It is a structured family of codes. The encoder first treats the transport block as the input constraint. It then chooses the LDPC "template" that best fits that constraint, so the same physical mechanism can support very different TBS and code-rate points without redefining the whole parity-check matrix every time.

The key idea is the separation between a small base graph and a large practical matrix. The base graph is the high-level structure that defines how variable nodes and check nodes are connected. Once the base graph is decided, the encoder does not build a brand-new graph from scratch. It "lifts" the base graph by a lifting size, so each base-graph entry expands into a block made of circularly shifted identity matrices. This is how the final H matrix becomes large enough for real payload sizes while keeping the structure regular and implementable.

After that, encoding becomes a constraint satisfaction problem in a very concrete form. You are looking for a codeword vector c that satisfies all parity checks, which is expressed as $H \cdot c = 0$. So the whole flow can be viewed as: pick the right structural template for the operating point, scale it up using a standardized lifting rule, and then generate parity bits so the resulting codeword satisfies the parity equations.



Now let me verbalize this diagram and it goes as follows

- **Overall flow (what the diagram is trying to teach)**
 - Start from a payload requirement (TBS) and a protection requirement (code rate)
 - Choose a standardized LDPC "template" (Base Graph)
 - Scale that template to the exact needed size (Lifting)
 - Build the parity-check matrix H from the lifted structure
 - Encode by finding a codeword c that satisfies the parity constraints
- **1) Input: TBS**
 - TBS is the anchor parameter
 - It represents the amount of transport data to be protected by LDPC
 - TBS is used together with code rate
 - Because "small vs large block" and "low vs high rate" drive different best structures
- **2) Base Graph selection (BG1 vs BG2)**
 - Base Graph is selected based on TBS and code rate
 - The figure emphasizes a region split where BG2 tends to cover smaller / different rate regions and BG1 covers large other regions
 - The Base Graph defines the mother structure size
 - BG1 has its own (M_b, N_b, k_b)
 - BG2 has its own (M_b, N_b, k_b)
 - Practical meaning
 - This step decides the "shape" of the LDPC code before any scaling happens
- **3) Lifting decision (Z_c, i_{LS})**
 - Once BG is fixed, choose the lifting size
 - Z_c is the expansion factor
 - Choose the lifting set index
 - i_{LS} selects which standardized lifting size entry is used
 - Practical meaning
 - This step turns "one base template" into "many usable code sizes" in a controlled way
- **4) Construct the full parity-check matrix H**
 - Build H by lifting the base graph
 - Each base-graph entry expands into a $Z_c \times Z_c$ block
 - Either a zero block or a shifted-identity type block
 - Resulting matrix dimensions scale with Z_c
 - For BG1: $(46 \cdot Z_c) \times (68 \cdot Z_c)$
 - For BG2: $(42 \cdot Z_c) \times (52 \cdot Z_c)$
 - Practical meaning
 - The base graph gives the pattern, lifting gives the real size
- **5) Codeword determination (encoding constraint)**
 - Encoding is framed as satisfying parity checks
 - Find codeword c such that $H \cdot c = 0$
 - Practical meaning
 - Parity bits are chosen so the final vector lies in the valid LDPC code space defined by H

Core Principle - Sparsity

Simply put, the final goal of LDPC is to find out parity bits that satisfy the following equation: $H \times c^T = 0 \text{ (modulo 2)}$. This equation is the foundation of Low-Density Parity-Check (LDPC) codes, ensuring that a codeword c adheres to the parity-check constraints defined by parity-check matrix H . The example in the image illustrates this principle through the encoding process, where the objective is to find parity bits p given information bits s , such that the resulting codeword c satisfies this equation. Let's elaborate on this as the core principle of LDPC, particularly in the context of 3GPP's 5G NR, and explore its broader significance.

- LDPC codes are defined by a **parity-check matrix (H)**. This matrix is "low-density," meaning it contains very few '1's and most '0's.
- Each row in H represents a **parity-check equation**, which is a simple XOR sum of a subset of the transmitted bits (codeword c) that must equal zero for a valid codeword.
- Each column corresponds to a specific bit in the codeword.
- The sparsity is crucial because it allows for efficient iterative decoding algorithms.

-20%

R\$ 303,20

-40%

R\$ 538,80

-40%

As a simple example, let's assume that we have a data $s = [s_1, s_2, s_3, s_4] = [1, 0, 1, 1]$, meaning there are 4 bits of data ($k = 4$) and to encode this to a codeword $c = [s_1, s_2, s_3, s_4, p_1, p_2, p_3, p_4]$. The final goal of LDPC is to find the p_1, p_2, p_3, p_4 to meet the condition $c^T = 0$ (modulo 2).

The complicated part of LDPC in real application (as in 5G) is about how to construct H matrix and how to decode the received LDPC into original message.

NOTE :If We Define the Precondition to $H \times c^T = 1$, It Would Not Work?

You're asking an interesting question about modifying the fundamental condition of LDPC codes from $H \times c^T = 0$ to $H \times c^T = 1$. Let's see why this change would not work in the context of LDPC codes, particularly as they are implemented in 3GPP's 5G NR, and what the implications would be.

Understanding the Standard Condition: $H \times c^T = 0$

In LDPC codes (and linear block codes in general), the condition $H \times c^T = 0$ (with operations in $GF(2)$, i.e., modulo 2) ensures that the codeword c satisfies all parity-check equations defined by the parity-check matrix H . This condition is fundamental for several reasons:

- **Parity Checks:** Each row of H represents a parity-check equation. For example, if a row of H has 1s in positions 1, 3, and 5, the corresponding equation is $c_1 + c_3 + c_5 = 0$, meaning the sum of those bits (modulo 2) must be 0 (an even number of 1s).
- **Error Detection:** At the receiver, if $H \times r^T \neq 0$ for a received vector r , it indicates errors, and the decoder uses this to correct them.
- **Systematic Structure:** The condition allows for systematic encoding, where the codeword c contains the original information s followed by parity bits that make the parity checks hold.

This setup ensures that the code has a well-defined structure, enabling efficient encoding, decoding, and error correction.

What Happens If We Change to $H \times c^T = 1$?

If we redefine the condition to $H \times c^T = 1$, we're fundamentally altering the rules of the code. Let's break down why this wouldn't work:

• Inconsistency with Linear Code Theory

- **Linear Codes:** LDPC codes are linear block codes, meaning the set of valid codewords forms a vector space (a subspace of $\{0,1\}^n$). For a linear code, the all-zero codeword $c = [0, 0, \dots, 0]$ must always be a valid codeword because it trivially satisfies $c \times 0^T = 0$.
- **With $H \times c^T = 1$:**
 - If $c = [0, 0, \dots, 0]$, then $H \times 0^T = 0$, which does not equal 1. So, the all-zero codeword would not be valid.
 - This violates the linearity property, as the set of codewords would no longer form a vector space (since the zero vector is not included).
- **Practical Impact:** Linearity is crucial for efficient encoding and decoding. Without it, many standard algorithms (like syndrome decoding or belief propagation in LDPC) would break down.

• Inconsistent Parity Checks

- **Standard Case ($H \times c^T = 0$):** Each row of H enforces a parity check where the sum of the bits (where H has 1s) must be 0 (modulo 2). For example, if a row of H is $[1, 0, 1, 0, 1, 0, 0, 0]$, the equation is $c_1 + c_3 + c_5 = 0$.
- **Modified Case ($H \times c^T = 1$):** Now, each row would require the sum of the bits to be odd (1 modulo 2). For the same row equation becomes $c_1 + c_3 + c_5 = 1$.
- **Problem:** If H has m rows, then $H \times c^T = 1$ means every parity check must result in 1 (an odd sum). This creates a highly constrained system:
 - The equations may not have a solution for all input data. For example, if two rows of H are identical (e.g., both enforce $c_1 + c_3 + c_5 = 1$), they cannot both be satisfied simultaneously because they would require the same sum to be both odd and odd, but there's only one sum.
 - In general, the system of equations $H \times c^T = [1, 1, \dots, 1]^T$ (an all-1 vector of length m) is over-constrained and has no solution, especially if the rows of H are not carefully designed to ensure solvability.

• Encoding Becomes Infeasible

- **Standard Encoding:** In LDPC (e.g., in 3GPP), we compute parity bits to satisfy $H \times c^T = 0$. This is straightforward because the equations are homogeneous (equal to 0), and we can solve for the parity bits systematically.
- **Modified Encoding:** With $H \times c^T = 1$, the equations become non-homogeneous (equal to 1). Solving for parity bits becomes much harder:
 - The system $H \times c^T = [1, 1, \dots, 1]^T$ may not have a solution, as mentioned above.
 - Even if a solution exists, it may not be unique, leading to ambiguity in encoding.
- **Practical Impact:** In 3GPP, encoding needs to be fast and deterministic for high-throughput applications (e.g., 5G's multi-carrier). An unsolvable or ambiguous system would make encoding impractical.

• Decoding Breaks Down

- **Standard Decoding:** LDPC decoding (e.g., Belief Propagation) relies on the syndrome $H \times r^T$. If $H \times r^T = 0$, the received r is a valid codeword. If not, the non-zero syndrome guides the iterative correction process.
- **Modified Decoding:** If the goal is $H \times r^T = 1$, the decoding process becomes problematic:
 - The syndrome would be computed as $H \times r^T \oplus [1, 1, \dots, 1]^T$, but the iterative message-passing algorithms (like Belief Propagation) are designed for the $H \times c^T = 0$ framework.
 - The Tanner graph (used for decoding) assumes that check nodes enforce even parity (sum = 0). Changing this to odd parity (sum = 1) for all checks disrupts the algorithm's convergence properties.
- **Practical Impact:** In 5G NR, decoding must be efficient and reliable. The modified condition would make error correction impossible or extremely inefficient.

• Loss of Error Detection Capability

- **Standard Case:** If a received vector r has errors, $H \times r^T \neq 0$, and the non-zero syndrome helps identify and correct errors.
- **Modified Case:** If the goal is $H \times r^T = 1$, the syndrome check becomes $H \times r^T \neq [1, 1, \dots, 1]^T$. But since the system is unsolvable, it's unclear how to interpret deviations from this condition, making error detection and correction unreliable.

Could It Work in a Different Context?

The condition $H \times c^T = 1$ resembles a non-homogeneous system, which isn't typical for linear block codes but could be interpreted in coding schemes:

- **Affine Codes:** Instead of a linear code (where the codewords form a vector space), you could define an affine code, where the codewords are an affine subspace (a linear subspace shifted by a fixed vector). This would mean $H \times c^T = b$, where b is a fixed vector.

-40%

R\$ 538,80

-20%

R\$ 303,20

-40%

zero vector (e.g., $[1, 1, \dots, 1]^T$). However:

- This is not how LDPC codes are defined in 3GPP or in standard practice.
- It loses the benefits of linearity, making encoding and decoding much harder.
- **Modified Parity Checks:** You could design a code where some parity checks result in 1 instead of 0, but this would require a new H matrix and a modified decoding algorithm, which isn't compatible with standard LDPC frameworks like those in 5G NR.

Practical Implications in 3GPP

In 3GPP's 5G NR, LDPC codes rely on $H \times c^T = 0$ for:

- **Efficient Encoding:** The quasi-cyclic structure of H (using base graphs and lifting) allows fast computation of parity bits.
- **Reliable Decoding:** Iterative decoding (Belief Propagation) is optimized for the $H \times c^T = 0$ condition, ensuring high throughput and low error rates.
- **Error Detection:** The syndrome check $H \times r^T = 0$ is critical for detecting and correcting errors in noisy channels.

Changing to $H \times c^T = 1$:

- Breaks the linearity of the code, making standard LDPC algorithms unusable.
- Makes encoding and decoding infeasible or inefficient, which is unacceptable for 5G's high-speed, reliable communication requirements.
- Disrupts the entire error correction framework, leading to unreliable data transmission.

Summary

The condition $H \times c^T = 1$ would not work for LDPC codes as defined in 3GPP or in standard practice because:

- It violates the linearity of the code, breaking fundamental properties.
- It creates an over-constrained system that often has no solution, making encoding impossible.
- It disrupts standard decoding algorithms, rendering error correction inefficient or infeasible.
- It loses the practical benefits (efficiency, reliability) that make LDPC codes suitable for 5G NR.

The $H \times c^T = 0$ condition is not arbitrary—it's a cornerstone of linear block codes like LDPC, ensuring they are practical and effective for world communication systems. If you'd like to explore alternative coding schemes where non-homogeneous conditions might apply, let me know!

LDPC Implementation in 5G - Structure

- **Quasi-Cyclic LDPC (QC-LDPC):** 3GPP uses a specific structure called QC-LDPC. This structure makes hardware implementation more efficient and manageable. The large parity-check matrix (H) is constructed from smaller sub-matrices which are either zero matrices or circulant permutation matrices (identity matrices with columns cyclically shifted).
- **Base Graphs (BG):** 3GPP defines two fundamental **Base Graphs**, BG1 and BG2.
 - **BG1:** Designed for larger data blocks and generally higher code rates.
 - **BG2:** Designed for smaller data blocks and generally lower code rates.
 - The choice between BG1 and BG2 depends on the transport block size and the target code rate.
- **Lifting:** The actual parity-check matrix used for encoding/decoding is generated by "lifting" the selected Base Graph (BG1 or BG2). This involves replacing each '1' in the base graph matrix with a specific circulant permutation matrix (based on shifts defined in standard) of size $Z \times Z$, and each '0' with a $Z \times Z$ zero matrix. The value ' Z ' is called the **lifting size**, and it's chosen from a set of predefined values based on the required code block length. This lifting process creates the larger, final QC-LDPC parity-check matrix.
- **Code Block Segmentation:** If a transport block (the data coming from higher layers) is very large, it's first segmented into smaller **code blocks**. A CRC (Cyclic Redundancy Check) is added to each code block before LDPC encoding. Each code block is then encoded independently using LDPC. This limits the maximum complexity of the LDPC encoder/decoder.

Encoding

While LDPC codes are defined by the parity-check matrix H , encoding isn't as straightforward as with some other codes. 3GPP defines efficient encoding procedures based on the structured nature (QC-LDPC) of the codes. Essentially, given the information bits, the encoder calculates the necessary parity bits such that the resulting codeword (information bits + parity bits) satisfies all the parity-check equations defined by H (i.e., $H \times c^T = 0$, where c is the codeword vector).

Rate Matching

Wireless systems need to adapt the coding rate (ratio of information bits to total transmitted bits) based on channel conditions. LDPC uses a flexible **rate matching** procedure.

- A base LDPC code (often low-rate) is generated.
- To achieve the desired code rate for transmission, some coded bits (usually parity bits, but sometimes information bits too for very low rates) are selected for transmission, while others are **punctured** (not transmitted) or **shortened**. For lower rates, bits might be repeated.
- 3GPP uses a **circular buffer** approach. All encoded bits (systematic/information and parity) are written into a buffer. Bits are then read out sequentially (and cyclically if needed) from this buffer for transmission, selecting the exact number of bits needed to match the allocated radio resources and target code rate.

Decoding

At the receiver, an **iterative decoding** algorithm is used, typically the **Belief Propagation** (also known as Sum-Product) algorithm or a variation like Min-Sum.

- This algorithm works on the **Tanner Graph** representation of the LDPC code (a bipartite graph connecting variable nodes representing codeword bits and check nodes representing parity-check equations).
- The decoder passes messages (probabilities or log-likelihood ratios) back and forth between variable nodes and check nodes along the edges of the Tanner graph.
- In each iteration, the nodes update their belief about the value of each received bit based on the messages received from their neighbors and the received signal strength for that bit.
- This process repeats for a fixed number of iterations or until the decoded bits form a valid codeword (i.e., satisfy all parity-check equations) or converge.
- The parallel nature of message passing across the sparse graph makes LDPC decoding suitable for high-speed hardware implementation.

What is QC-LDPC?

QC-LDPC is a specific type of Low-Density Parity-Check (LDPC) code where the parity-check matrix (H) has a special, structured form. Instead of being an arbitrary sparse matrix, it's constructed from smaller square blocks, each of which is either a **zero matrix** or a **circulant permutation matrix (CPM)**.

-20%

R\$ 303,20

-40%

R\$ 538,80

-40%

Key Concepts

1. LDPC Recap: Remember that LDPC codes are defined by a sparse parity-check matrix H . A vector c is a valid codeword if $H \cdot c^T$ Sparsity allows for efficient iterative decoding.

2. Cyclic Codes: In a *strictly* cyclic code, if you cyclically shift a valid codeword, you get another valid codeword. QC-LDPC is a generalization.

3. Circulant Permutation Matrix (CPM): This is the fundamental building block. A CPM is a square matrix (say, size $Z \times Z$) where c is a cyclic shift of the row above it. Crucially, each row and column has exactly one '1' and the rest '0's. They are essentially cyclically identity matrices.

- The **identity matrix** (I) is a CPM (zero shift).
- A **cyclically shifted identity matrix** is a CPM. For example, shifting the 3×3 identity matrix I_3 right by one position gives: I_3

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ Shifted}(I_3, 1) = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

- The **zero matrix** (all zeros) is often considered trivially in this context as well, representing the absence of connections in the graph.

4. Quasi-Cyclic Structure: The "Quasi" part means the *entire* H matrix isn't necessarily cyclic, but it *is* composed of these CPM blocks (and zero blocks). If you divide the H matrix into $Z \times Z$ blocks, each block is a CPM or a zero matrix.

How is the H Matrix Constructed?

Often, a QC-LDPC matrix H is defined by:

1. A Lifting Factor (or Expansion Factor) Z : This defines the size ($Z \times Z$) of the circulant blocks.

2. A Base Matrix (or Model Matrix) B : This is a smaller matrix whose entries tell you *which* CPM to place in the corresponding block position in the final H matrix.

- An entry s (where $s \geq 0$) in the base matrix means: place the $Z \times Z$ identity matrix cyclically shifted by s positions in that block location.
- A special entry (e.g., -1 or ∞ , depending on convention) means: place the $Z \times Z$ zero matrix in that block location.

If the Base Matrix B has dimensions $M_b \times N_b$, and the lifting factor is Z , the final QC-LDPC matrix H will have dimensions $(M_b * Z) \times (N_b * Z)$.

Example

Let's construct a small QC-LDPC matrix H .

Choose **Lifting Factor $Z = 3$** .

Define a **Base Matrix B** (size 2×4): Base Matrix refers to the matrix representation of what is called 'Base Graph' in 3GPP.

$$B = \begin{bmatrix} 0 & 2 & -1 & 1 \\ 1 & -1 & 2 & 0 \end{bmatrix}$$

This is just an example of the simple Base Matrix. This is a **Base Matrix (B)** used to define the structure of a **Quasi-Cyclic LDPC (QC-LDPC)** parity-check matrix (H).

This matrix doesn't directly contain message data. Instead, each number tells you how to construct a *block* within the larger, final parity-check matrix H . You need a **Lifting Factor (Z)** (which was $Z=3$ in the previous example) to interpret these numbers:

- Non-negative numbers ($0, 1, 2$):** These numbers represent a **cyclic shift** applied to a $Z \times Z$ **identity matrix**.
 - 0 : Represents the $Z \times Z$ identity matrix with **no shift** (P^0).
 - 1 : Represents the $Z \times Z$ identity matrix **cyclically shifted right by 1** position (P^1).
 - 2 : Represents the $Z \times Z$ identity matrix **cyclically shifted right by 2** positions (P^2).
 - In general, a non-negative number s represents the $Z \times Z$ identity matrix shifted s times (P^s).
- Negative number (-1):** As the text below the matrix in the image notes, the convention here is that -1 represents the **$Z \times Z$ zero matrix** (0_Z). This means this block position in the final H matrix will consist entirely of zeros. (Sometimes other conventions like infinity are used for the zero matrix).

Now, we need the 3×3 circulant permutation matrices corresponding to the shifts $0, 1$, and 2 , plus the 3×3 zero matrix:

P^0 (Identity, shift 0):

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

P^1 (Shift right by 1):

$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

P^2 (Shift right by 2):

$$\begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

0_3 (Zero matrix):

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

-27%



Now, substitute these blocks into H according to the base matrix B:

$$H = \begin{bmatrix} \text{Block Col 1} & \text{Block Col 2} & \text{Block Col 3} & \text{Block Col 4} \\ p^0 & p^2 & \theta_3 & p^1 \\ \text{Block Row 1} \\ \hline p^1 & \theta_3 & p^2 & p^0 \\ \text{Block Row 2} \end{bmatrix}$$

Writing this out fully:

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

This resulting H matrix is:

- **Sparse:** It has relatively few '1's.
- **Quasi-Cyclic:** It's built from 3x3 circulant (or zero) blocks. The dimensions are $(2 * 3) \times (4 * 3) = 6 \times 12$.

Why Use QC-LDPC?

- **Hardware Implementation Efficiency:** The regular structure is extremely beneficial for hardware (like FPGAs or ASICs). Enc and especially decoding can be implemented using parallel shift-register-based architectures, making them fast and efficient. Memory addressing is simplified.
- **Efficient Encoding:** While general LDPC encoding can be complex, the structure of QC-LDPC allows for more efficient encoding algorithms (often based on linear feedback shift registers).
- **Compact Representation:** You only need to store the base matrix and the lifting factor Z to define a potentially very large H. This is how standards like 5G NR and Wi-Fi define their LDPC codes – specifying base graphs (matrices) and allowed lifting size.
- **Scalability:** A single base graph can generate a family of codes with different lengths just by changing the lifting factor Z.

In essence, QC-LDPC codes offer the excellent error-correction performance associated with LDPC codes while having a structure that is much more practical and efficient to implement in real-world communication systems.

What is Lifting ?

Lifting is the process used in QC-LDPC code construction to **expand a small Base Graph (represented by a Base Matrix B) into a final, much larger parity-check matrix H**.

It works like this: You start with a **Base Matrix B** (like the 2x4 example B we discussed) and choose a **Lifting Factor Z** (also called L Size or Expansion Factor), which is a positive integer (e.g., Z=3 in the previous example). Then, you replace **each entry** in the Base with a **Z x Z block matrix** according to these rules:

- If an entry in B is a non-negative integer s (representing a shift), you replace it with the Z x Z identity matrix cyclically shifted (p^s).
- If an entry in B represents the zero block (e.g., -1 in the convention used), you replace it with the Z x Z zero matrix (0_Z).

The result is the final parity-check matrix H. If the Base Matrix B has dimensions M_b x N_b, the lifted matrix H will have dimensions (M_b x Z) x (N_b x Z).

NOTE: The 2x4 Base Matrix B in the example of previous section was lifted with Z=3 to create the 6x12 parity-check matrix H.

Why is Lifting Needed?

Lifting is a crucial technique used in modern communication standards (like 5G, Wi-Fi) for several important reasons:

- **Code Scalability and Flexibility:** This is arguably the most significant benefit. Communication systems need to handle data of many different sizes. Instead of designing a completely new LDPC code (and a new H matrix) for every possible size, lifting allows a single Base Graph to generate a whole family of codes with different lengths. By simply choosing a different Lifting Factor Z from a predefined set, you can create a valid LDPC code tailored to the required block size.
- **Compact Code Definition:** Standards like 3GPP don't need to specify enormous H matrices for every scenario. They only need to define a few relatively small Base Graphs (like BG1 and BG2) and a set of allowed Lifting Factors (Z values). This makes the standards much more compact and manageable.
- **Maintaining Good Code Properties:** The Base Graphs are carefully designed to have good structural properties (like large girth in their Tanner graph representation) which are important for the performance of the iterative LDPC decoder. The lifting process is designed to preserve these beneficial properties in the final, larger H matrix.
- **Efficient Hardware/Software Implementation:** The highly structured nature of the lifted matrix H (composed of repeating shifted identity matrices and zero matrices) lends itself well to efficient implementation. Decoders can leverage this structure for parallel processing, shift registers, and optimized memory access patterns. The underlying logic can often be reused regardless of the specific lifting factor Z chosen.
- **System Adaptability:** Wireless communication requires constant adaptation to changing channel conditions and data requirements. Lifting provides an efficient mechanism for the physical layer to generate the appropriate LDPC code (with the right length and corresponding parity-check matrix H) on the fly by selecting the necessary lifting factor Z based on the current transmission parameters.

In summary, lifting provides an elegant and efficient way to generate families of high-performing LDPC codes of various lengths from a set of core designs (the Base Graphs), which is essential for the flexibility and efficiency required in modern communication systems.

Lifting Table for 5G LDPC

The process of selecting the lifting size Z_c in the context of LDPC is a critical step in the encoding procedure for 5G New Radio (NR). It took me several years for me to understand the overall concept of the selection of a specific Lifting size. The lifting size Z_c determines how a base graph (a small, predefined matrix) is expanded into a larger parity-check matrix H that is used for encoding. Let's break down the selection process based on the document.

Key Parameters and Context

- **LDPC Base Graphs:** The document mentions two LDPC base graphs:
 - **Base Graph 1:** Used when the code block length K ≤ 8448. It has 46 rows and 68 columns.

- **Base Graph 2:** Used when $K > 8448$. It has 42 rows and 52 columns.
- **Input Bit Sequence:** The input to the encoder is a bit sequence $c_0, c_1, c_2, \dots, c_{K-1}$, where K is the number of bits to encode.
- **Output After Encoding:** The encoded bits are denoted as $d_0, d_1, d_2, \dots, d_{N-1}$, where N is the total number of encoded bits.
- **Lifting Size Z_c :** This is a scaling factor that "lifts" the base graph into a larger parity-check matrix $\mathbf{H}$. The value of Z_c is chosen from a predefined set in a table (referred to as Table 5.3.2-1 in the 3GPP document).
- **Dimensions After Lifting:**
 - For Base Graph 1: $N = 66Z_c$, since the base graph has 66 columns.
 - For Base Graph 2: $N = 50Z_c$, since the base graph has 50 columns.

The lifting size Z_c must be chosen such that the base graph can accommodate the input bit sequence of length K , and the resulting parity-check matrix $\mathbf{H}$ can be used for encoding.

Step-by-Step Process for Selecting Z_c

The selection of Z_c is described in **Step 1** and **Step 2** of the encoding procedure in the document. Let's go through these steps in detail.

Step 1: Find the Set with Index i_{LS} in Table 5.3.2-1 that Contains Z_c

- The 3GPP standard defines a table (Table 5.3.2-1) that contains sets of possible lifting sizes Z_c . Each set is indexed by i_{LS} , and contain different values of Z_c .
- The lifting size Z_c is not explicitly given in the problem statement but is typically determined based on the input length K and the base graph being used. The goal of this step is to identify which set in the table contains the appropriate Z_c .

Step 2: For $k = 22Z_c$ to $K-1$, Perform the Following

- The value $k = 22Z_c$ represents the number of information bits that the base graph can handle after lifting by Z_c . This comes from the structure of the base graph:
 - Both LDPC base graphs have 22 columns dedicated to information bits (the remaining columns are for parity bits).
 - When the base graph is lifted by Z_c , each column corresponds to Z_c bits, so the number of information bits that can be encoded is $22Z_c$.
- The condition $k = 22Z_c$ to $K-1$ checks whether the lifted base graph can accommodate the input bit sequence of length K . In other words, we need $22Z_c \geq K$ to ensure the base graph can handle all K bits.
- **If $c_k \neq \text{NULL}$:**
 - If the bit c_k at position k (for k ranging from $22Z_c$ to $K-1$) is not a NULL bit (i.e., it's an actual information bit), then the Z_c is too small to accommodate all K bits.
 - In this case, set:
 - $d_{k-22Z_c} = c_k$: This means the bits beyond $22Z_c$ are copied into the output sequence d , effectively shortening the input sequence to fit the base graph.
- **Else:**
 - If $c_k = \text{NULL}$, the bit at position k is a filler bit (used for padding).
 - Set:
 - $c_k = 0$: Replace the NULL bit with 0.
 - $d_{k-22Z_c} = \text{NULL}$: Mark the corresponding position in the output as NULL.

Implicit Selection of Z_c

- The document doesn't explicitly show the selection of Z_c , but the process is implied through the condition $22Z_c \geq K$.
- To select Z_c :
 - Compute the minimum Z_c such that $22Z_c \geq K$.
 - From the set of possible Z_c values in Table 5.3.2-1, choose the smallest Z_c that satisfies this condition.
- For example:
 - If $K = 5000$, then $22Z_c \geq 5000$, so $Z_c \geq \lceil 5000 / 22 \rceil = 228$.
 - You would then look in Table 5.3.2-1 for the smallest $Z_c \geq 228$ in the appropriate set (determined by i_{LS}).

Additional Details

- **Table 5.3.2-1:** This table contains sets of lifting sizes. Typically, in 3GPP LDPC, the lifting sizes are grouped into sets like $\{2, 4, \dots\}$, $\{3, 6, 12, 24, \dots\}$, etc., and the set index i_{LS} is determined based on the code block length K and the base graph.
- **Base Graph Selection:**
 - If $K \leq 8448$, use Base Graph 1.
 - If $K > 8448$, use Base Graph 2.
 - The choice of base graph affects the number of information columns (always 22 for both graphs) and the total number of columns (66 for Base Graph 1, 50 for Base Graph 2), which in turn affects N .
- **Matrix $\mathbf{H}$:** The parity-check matrix $\mathbf{H}$ is constructed by lifting the base graph $\mathbf{H}_{BG}$ using Z_c . Each element in $\mathbf{H}_{BG}$ is replaced as:
 - A 0 in $\mathbf{H}_{BG}$ becomes an all-zero matrix of size $Z_c \times Z_c$.
 - A 1 in $\mathbf{H}_{BG}$ becomes a circular permutation matrix $\mathbf{I}(P_{i,j})$ of size $Z_c \times Z_c$, where $P_{i,j}$ is a shift value determined by the base graph and Z_c .

Summary of Z_c Selection

- **Determine the Base Graph:**
 - Use Base Graph 1 if $K \leq 8448$.
 - Use Base Graph 2 if $K > 8448$.
- **Compute the Minimum Z_c :**
 - Calculate the smallest Z_c such that $22Z_c \geq K$.
- **Select Z_c from Table 5.3.2-1:**
 - Look up Table 5.3.2-1 and find the set indexed by i_{LS} .
 - Choose the smallest Z_c in that set that satisfies $22Z_c \geq K$.
- **Handle Padding or Shortening:**
 - If $K < 22Z_c$, pad the input with NULL bits (which are later set to 0).
 - If $K > 22Z_c$, shorten the input by copying bits into the output sequence d .

Example

Suppose $K = 5000$, and we're using Base Graph 1 (since $K \leq 8448$):

- **Compute Z_c :**
 - $22Z_c \geq 5000$
 - $Z_c \geq \lceil 5000 / 22 \rceil = 228$.

- Assume Table 5.3.2-1 has a set with Z_c values like {208, 224, 240, 256, ...}.
- The smallest $Z_c \geq 228$ is 240.
- So, $Z_c = 240$, and the number of information bits the base graph can handle is $22 \times 240 = 5280$, which is greater than $K = 50$ remaining $5280 - 5000 = 280$ bits are padded with NULLs (later set to 0).

Inter-relation among Table 5.3.2-1, Table 5.3.2-2 and Table 5.3.2-3

They are interrelated because they collectively define the structure and parameters needed to construct the parity-check matrix $\mathbf{H}$, which is used for encoding data in the LDPC process. In short, the tables are tightly interrelated as part of the LDPC encoding process:

- **Table 5.3.2-1** provides the foundational parameter Z_c and the set index i_{LS} .
- **Tables 5.3.2-2 and 5.3.2-3** use Z_c and i_{LS} to define the structure of the base graph and the shift values needed to construct the parity-check matrix $\mathbf{H}$.
- The choice between Tables 5.3.2-2 and 5.3.2-3 depends on the code block length K , but both rely on the parameters from Table 5.3.2-1 to complete the encoding process.

Here's how they connect:

Table 5.3.2-1 Provides the Lifting Size Z_c , Which is Used by Tables 5.3.2-2 and 5.3.2-3

- **Connection:** The lifting size Z_c from Table 5.3.2-1 is a key parameter needed to interpret the shift values $V_{i,j}$ in Tables 5.3.2-2 and 5.3.2-3.
- **Details:**
 - In the LDPC encoding process (as described in the document), the first step is to select a lifting size Z_c from Table 5.3.2-1 on the code block length K . This involves finding the set index i_{LS} and the smallest Z_c in that set such that $22Z_c \geq K$.
 - The selected Z_c determines the size of the submatrices used to lift the base graph $\mathbf{H}_{BG}$ into the full parity-check matrix $\mathbf{H}$. Specifically, each element in $\mathbf{H}_{BG}$ is replaced by a $Z_c \times Z_c$ submatrix.
 - The shift values $V_{i,j}$ in Tables 5.3.2-2 and 5.3.2-3 are used to compute the actual cyclic shift $P_{i,j}$ for each submatrix, where $\text{mod}(V_{i,j}, Z_c)$. This means Z_c from Table 5.3.2-1 directly affects how the shift values in Tables 5.3.2-2 and 5.3.2-3 are applied.

Tables 5.3.2-2 and 5.3.2-3 Depend on the Set Index i_{LS} from Table 5.3.2-1

- **Connection:** The shift values $V_{i,j}$ in Tables 5.3.2-2 and 5.3.2-3 are specified for each set index i_{LS} , which is determined when Z_c is selected from Table 5.3.2-1.
- **Details:**
 - Table 5.3.2-1 is organized into sets, each with an index i_{LS} . When Z_c is selected, the corresponding i_{LS} is also determined.
 - Tables 5.3.2-2 and 5.3.2-3 provide $V_{i,j}$ values for each position (i, j) where $\mathbf{H}_{BG}$ is 1, and these values are given for each i_{LS} . This means that once i_{LS} is chosen from Table 5.3.2-1, it dictates which set of $V_{i,j}$ values to use from Tables 5.3.2-2 and 5.3.2-3, depending on the base graph.

Tables 5.3.2-2 and 5.3.2-3 Define the Base Graphs, Which Are Lifted Using Z_c

- **Connection:** Tables 5.3.2-2 and 5.3.2-3 provide the structure of the base graphs ($\mathbf{H}_{BG}$) and the shift values $V_{i,j}$, which are used in conjunction with Z_c to construct the final parity-check matrix $\mathbf{H}$.
- **Details:**
 - The base graph $\mathbf{H}_{BG}$ (from Table 5.3.2-2 for Base Graph 1 or Table 5.3.2-3 for Base Graph 2) is a small binary matrix that defines the connections between check nodes and variable nodes.
 - To create the full parity-check matrix $\mathbf{H}$, each element in $\mathbf{H}_{BG}$ is replaced as follows:
 - A 0 in $\mathbf{H}_{BG}$ is replaced by an all-zero matrix of size $Z_c \times Z_c$.
 - A 1 in $\mathbf{H}_{BG}$ is replaced by a circular permutation matrix $\mathbf{I}(P_{i,j})$ of size $Z_c \times Z_c$, where $P_{i,j} = \text{mod}(V_{i,j}, Z_c)$.
 - Here, Z_c (from Table 5.3.2-1) determines the size of the submatrices, and $V_{i,j}$ (from Table 5.3.2-2 or 5.3.2-3) determines the shift for each submatrix.

Base Graph Selection Affects Which Table (5.3.2-2 or 5.3.2-3) is Used

- **Connection:** The choice between Table 5.3.2-2 and Table 5.3.2-3 depends on the code block length K , but both tables use the parameters Z_c and i_{LS} from Table 5.3.2-1.
- **Details:**
 - If $K \leq 8448$, Base Graph 1 is used, so Table 5.3.2-2 provides the structure of $\mathbf{H}_{BG}$ and the shift values $V_{i,j}$.
 - If $K > 8448$, Base Graph 2 is used, so Table 5.3.2-3 provides the structure of $\mathbf{H}_{BG}$ and the shift values $V_{i,j}$.
 - In both cases, the Z_c and i_{LS} selected from Table 5.3.2-1 are used to interpret the $V_{i,j}$ values and construct $\mathbf{H}$.

Summary of Interrelations

- **Table 5.3.2-1** is the starting point: It provides the lifting size Z_c and the set index i_{LS} , which are determined based on the code block length K .
- **Tables 5.3.2-2 and 5.3.2-3** depend on Z_c and i_{LS} :
 - The set index i_{LS} from Table 5.3.2-1 determines which set of $V_{i,j}$ values to use from Table 5.3.2-2 (for Base Graph 1) or Table 5.3.2-3 (for Base Graph 2).
 - The lifting size Z_c from Table 5.3.2-1 is used to compute the actual shift $P_{i,j} = \text{mod}(V_{i,j}, Z_c)$ and to determine the size of the submatrices in the lifted matrix $\mathbf{H}$.
- **Tables 5.3.2-2 and 5.3.2-3** are alternatives based on the base graph:
 - They provide the structure of $\mathbf{H}_{BG}$ and the shift values $V_{i,j}$, but only one table is used at a time, depending on whether Base Graph 1 or Base Graph 2 is selected (based on K).

Flow of Dependency

- **Determine K :** The code block length K determines which base graph to use (Base Graph 1 if $K \leq 8448$, Base Graph 2 if $K > 8448$).
- **Select Z_c and i_{LS} from Table 5.3.2-1:** Based on K , choose the smallest Z_c such that $22Z_c \geq K$, and note the corresponding set index i_{LS} .
- **Choose the Base Graph Table:**
 - If Base Graph 1, use Table 5.3.2-2.
 - If Base Graph 2, use Table 5.3.2-3.
- **Use i_{LS} to Get $V_{i,j}$:** From the chosen table (5.3.2-2 or 5.3.2-3), look up the $V_{i,j}$ values corresponding to the set index i_{LS} .
- **Construct $\mathbf{H}$:** Use Z_c to determine the size of the submatrices, and use $V_{i,j}$ to compute the shifts $P_{i,j}$, thereby lifting $\mathbf{H}_{BG}$ into the final parity-check matrix $\mathbf{H}$.

< 38.212-Table 5.3.2-1: Sets of LDPC lifting size Z >

The structure, role of this table can be described as below :

-27%



-53%

-54%

-55%

NOVO

- **Purpose:** This table provides sets of possible lifting sizes Z_c , which are used to scale the base graph into a larger parity-check H .
- **Structure:** The table is organized into sets, each indexed by i_{LS} (set index). Each set contains a range of Z_c values. For example, might include values like $\{2, 4, 8, 16, \dots\}$, $\{3, 6, 12, 24, \dots\}$, etc.
- **Role in LDPC:** The lifting size Z_c determines the size of the submatrices used to expand the base graph H_{BG} . It is selected based on the code block length K (as explained in my previous response) to ensure the base graph can accommodate the input bits.

Set index (i_{LS})	Set of lifting sizes (Z)
0	$\{2, 4, 8, 16, 32, 64, 128, 256\}$
1	$\{3, 6, 12, 24, 48, 96, 192, 384\}$
2	$\{5, 10, 20, 40, 80, 160, 320\}$
3	$\{7, 14, 28, 56, 112, 224\}$
4	$\{9, 18, 36, 72, 144, 288\}$
5	$\{11, 22, 44, 88, 176, 352\}$
6	$\{13, 26, 52, 104, 208\}$
7	$\{15, 30, 60, 120, 240\}$

NOTE : This table can be generated by the rule : $\text{base_Z} \times 2^j$, where base_Z = the first elements in the set, $j = [0 \dots 7]$

Set index (i_{LS})	base_Z	Set of lifting sizes (Z)
0	2	$\{2 \cdot 2^0 = 2, 2 \cdot 2^1 = 4, 2 \cdot 2^2 = 8, 2 \cdot 2^3 = 16, 2 \cdot 2^4 = 32, 2 \cdot 2^5 = 64, 2 \cdot 2^6 = 128, 2 \cdot 2^7 = 256\}$
1	3	$\{3 \cdot 2^0 = 3, 3 \cdot 2^1 = 6, 3 \cdot 2^2 = 12, 3 \cdot 2^3 = 24, 3 \cdot 2^4 = 48, 3 \cdot 2^5 = 96, 3 \cdot 2^6 = 192, 3 \cdot 2^7 = 384\}$
2	5	$\{5 \cdot 2^0 = 5, 5 \cdot 2^1 = 10, 5 \cdot 2^2 = 20, 5 \cdot 2^3 = 40, 5 \cdot 2^4 = 80, 5 \cdot 2^5 = 160, 5 \cdot 2^6 = 320, 5 \cdot 2^7 = 640\}$
3	7	$\{7 \cdot 2^0 = 7, 7 \cdot 2^1 = 14, 7 \cdot 2^2 = 28, 7 \cdot 2^3 = 56, 7 \cdot 2^4 = 112, 7 \cdot 2^5 = 224, 7 \cdot 2^6 = 448, 7 \cdot 2^7 = 896\}$
4	9	$\{9 \cdot 2^0 = 9, 9 \cdot 2^1 = 18, 9 \cdot 2^2 = 36, 9 \cdot 2^3 = 72, 9 \cdot 2^4 = 144, 9 \cdot 2^5 = 288, 9 \cdot 2^6 = 576, 9 \cdot 2^7 = 1152\}$
5	11	$\{11 \cdot 2^0 = 11, 11 \cdot 2^1 = 22, 11 \cdot 2^2 = 44, 11 \cdot 2^3 = 88, 11 \cdot 2^4 = 176, 11 \cdot 2^5 = 352, 11 \cdot 2^6 = 704, 11 \cdot 2^7 = 1408\}$
6	13	$\{13 \cdot 2^0 = 13, 13 \cdot 2^1 = 26, 13 \cdot 2^2 = 52, 13 \cdot 2^3 = 104, 13 \cdot 2^4 = 208, 13 \cdot 2^5 = 416, 13 \cdot 2^6 = 832, 13 \cdot 2^7 = 1664\}$
7	15	$\{15 \cdot 2^0 = 15, 15 \cdot 2^1 = 30, 15 \cdot 2^2 = 60, 15 \cdot 2^3 = 120, 15 \cdot 2^4 = 240, 15 \cdot 2^5 = 480, 15 \cdot 2^6 = 960, 15 \cdot 2^7 = 1920\}$

In the 3GPP TS 38.212 standard, the lifting sizes are capped at 384. The table above follows the mathematical "doubling" rule strictly. In practical 5G implementation, values exceeding 384 are typically not used for these specific sets.

Removing the capped number, we can have the following table which is same as 38.212 - Table 5.3.2-1

Set index (i_{LS})	base_Z	Set of lifting sizes (Z)
0	2	$\{2, 4, 8, 16, 32, 64, 128, 256\}$
1	3	$\{3, 6, 12, 24, 48, 96, 192, 384\}$
2	5	$\{5, 10, 20, 40, 80, 160, 320\}$
3	7	$\{7, 14, 28, 56, 112, 224\}$
4	9	$\{9, 18, 36, 72, 144, 288\}$
5	11	$\{11, 22, 44, 88, 176, 352\}$
6	13	$\{13, 26, 52, 104, 208\}$
7	15	$\{15, 30, 60, 120, 240\}$

< 38.212-Table 5.3.2-2: LDPC base graph 1 (H_{BG}) and its parity check matrices ($V_{i,j}$) >

The structure, role of this table can be described as below :

- **Purpose:** This table defines the structure of LDPC Base Graph 1 and the associated shift values $V_{i,j}$.
- **Structure:**
 - **Base Graph 1 (H_{BG}):** A binary matrix with 46 rows (check nodes) and 68 columns (variable nodes). The first 22 columns correspond to information bits, and the remaining columns correspond to parity bits.
 - **Elements:** The table specifies the positions (i, j) where H_{BG} has a value of 1 (indicating a connection between a check node i and a variable node j). All other positions are 0.
 - **Shift Values ($V_{i,j}$):** For each position (i, j) where H_{BG} is 1, the table provides a shift value $V_{i,j}$. This value depends on the index i_{LS} (from Table 5.3.2-1) and is used to define the circular permutation matrix that replaces the 1 in the lifted matrix.
- **Role in LDPC:** Base Graph 1 is used when the code block length $K \leq 8448$. The shift values $V_{i,j}$ are used to construct the parity-check matrix H by determining the cyclic shifts in the submatrices.

-27%



-53%

-54%

-55%

NOVO

H_{BG}		$V_{i,j}$								H_{BG}		$V_{i,j}$							
Row index i	Column index j	Set index i_{LS}								Row index i	Column index j	Set index i_{LS}							
		0	1	2	3	4	5	6	7			0	1	2	3	4	5	6	7
0	0	250	307	73	223	211	294	0	135	15	1	96	2	290	120	0	348	6	138
	1	69	19	15	16	168	118	0	227		10	65	210	60	131	183	15	81	220
	2	226	50	103	94	188	167	0	126		13	63	318	130	209	108	81	182	173
	3	156	369	49	91	186	330	0	134		18	75	55	184	209	68	176	53	142
	5	100	181	240	74	219	207	0	84		25	179	269	51	81	64	113	46	49
	6	10	216	39	10	4	165	0	83		37	0	0	0	0	0	0	0	0
	9	59	317	15	0	29	243	0	53	16	1	64	13	69	154	270	160	88	78
	10	229	288	162	205	144	250	0	225		3	49	338	140	164	13	293	198	152
	11	110	109	215	216	116	1	0	205		11	49	57	45	43	99	332	160	84
	12	191	17	164	21	216	339	0	128		20	51	289	115	189	54	331	122	5
	13	9	357	133	215	115	201	0	75		22	154	57	300	101	0	114	182	205
	15	195	215	298	14	233	53	0	135		38	0	0	0	0	0	0	0	0
	16	23	106	110	70	144	347	0	217	17	0	7	260	257	56	153	110	91	183
	18	190	242	113	141	95	304	0	220		14	164	303	147	110	137	228	184	112
	19	35	180	16	198	216	167	0	90		16	59	81	128	200	0	247	30	106
	20	239	330	189	104	73	47	0	105		17	1	358	51	83	0	116	3	219
	21	31	346	32	81	261	188	0	137		21	144	375	228	4	162	190	155	129
	22	1	1	1	1	1	1	0	1		39	0	0	0	0	0	0	0	0
	23	0	0	0	0	0	0	0	0	18	1	42	130	260	199	161	47	1	183
	0	2	76	303	141	179	77	22	96		12	233	163	294	110	151	286	41	215
	2	239	76	294	45	162	225	11	236		13	8	280	291	200	0	246	167	180
	3	117	73	27	151	223	96	124	136		18	155	132	141	143	241	181	68	143
	4	124	288	261	46	256	338	0	221		19	147	4	295	186	144	73	148	14
	5	71	144	161	119	160	268	10	128		40	0	0	0	0	0	0	0	0
	7	222	331	133	157	76	112	0	92	19	0	60	145	64	8	0	87	12	179
	8	104	331	4	133	202	302	0	172		1	73	213	181	6	0	110	6	108
	9	173	178	80	87	117	50	2	56		7	72	344	101	103	118	147	168	159
	11	220	295	129	206	109	167	16	11		8	127	242	270	198	144	258	184	138
	12	102	342	300	93	15	253	60	189		10	224	197	41	8	0	204	191	166
	14	109	217	76	79	72	334	0	95		41	0	0	0	0	0	0	0	0
	15	132	99	266	9	152	242	6	85	20	0	151	187	301	105	265	89	6	77
	16	142	354	72	118	158	257	30	153		3	186	206	162	210	81	65	12	187
	17	155	114	83	194	147	133	0	87		9	217	264	40	121	90	155	15	203
	19	255	331	260	31	156	9	168	163		11	47	341	130	214	144	244	5	167
	21	28	112	301	187	119	302	31	216		22	160	59	10	183	228	30	30	130
	22	0	0	0	0	0	0	105	0		42	0	0	0	0	0	0	0	0
	23	0	0	0	0	0	0	0	0	21	1	249	205	79	192	64	162	6	197
	24	0	0	0	0	0	0	0	0		5	121	102	175	131	46	264	86	122
	0	106	205	68	207	258	226	132	189		16	109	328	132	220	266	346	96	215
	1	111	250	7	203	167	35	37	4		20	131	213	283	50	9	143	42	65
	2	185	328	80	31	220	213	21	225		21	171	97	103	106	18	109	199	216
	4	63	332	280	176	133	302	160	151		43	0	0	0	0	0	0	0	0
	5	117	256	38	180	243	111	4	236	22	0	64	30	177	53	72	280	44	25
	6	93	161	227	186	202	265	149	117		12	142	11	20	0	189	157	58	47
	7	229	267	202	95	218	128	48	179		13	188	233	55	3	72	236	130	126
	8	177	160	200	153	63	237	38	92		17	158	22	316	148	257	113	131	178
	9	95	63	71	177	0	264	122	24		44	0	0	0	0	0	0	0	0
	10	39	129	106	70	3	127	195	68		1	156	24	249	88	180	18	45	185
	13	142	200	295	77	74	110	155	6	23	2	147	89	50	203	0	6	18	127
	14	225	88	283	214	229	286	28	101		10	170	61	133	168	0	181	132	117
	15	225	53	301	77	0	125	85	33		18	152	27	105	122	165	304	100	199
	17	245	131	194	198	218	131	47	96		45	0	0	0	0	0	0	0	0
	18	205	240	246	117	269	163	179	125		0	112	298	289	46	236	38	9	32
	19	251	205	230	223	200	210	42	67		3	86	158	280	157	199	170	125	178
	20	117	13	276	90	234	7	66	230	24	4	236	235	110	64	0	249	191	2
	24	0	0	0	0	0	0	0	0		11	116	339	187	193	268	288	28	156
	25	0	0	0	0	0	0	0	0		22	222	234	281	124	0	164	6	58
	0	121	276	220	201	187	97	4	128		46	0	0	0	0	0	0	0	0
	1	89	87	208	18	145	94	6	23		1	23	72	172	1	205	279	4	27
	3	84	0	30	165	166	49	33	162		6	136	17	295	166	0	255	74	141
	4	20	275	197	5	108	279	113	220	25	7	116	383	98	65	0	111	16	11
	6	150	199	61	45	82	139	49	43		14	182	312	46	81	183	54	28	181
	7	131	153	175	142	132	166	21	188		47	0	0	0	0	0	0	0	0
	8	243	56	79	16	197	91	6	96		0	195	71	270	107	0	325	21	163
	10	136	132	281	34	41	106	151	1		2	243	81	110	176	0	326	142	131
	11	89	305	303	155	162	246	83	216		4	215	76	318	212	0	226	192	169
	12	246	231	253	213	57	345	154	22	26	15	61	136	67	127	277	99	197	98
	13	219	341	164	147	36	269	87	24		48	0	0	0	0	0	0	0	0
	14	211	212	53	69	115	185	5	167		1	25	164	210	208	45	91	98	165
	16	240	304	44	96	242	249	92	200		6	104	164	29	141	36	326	140	232
	17	76	300	28	74	165	215	173	32		8	164	101	304	174	72	268	22	9
	18	244	271	77	69	0	143	120	235		49	0	0	0	0	0	0	0	0
	20	144	39	319	30	113	121	2	172	27	0	128	222	11	146	275	102	4	32
	21	12	357	68	158	108	121	142	219		4	165	19	293	153	0	1	1	43
	22	1	1	1	1	1	1	0	1		19	181	244	50	217	155	40	40	200
	25	0	0	0	0	0	0	0	0										

-18%



-26%

6	16	28	241	201	182	254	295	207	210	31	10	73	229	23	130	90	16	88	199
	21	123	51	267	130	79	258	161	180		13	120	280	105	210	252	95	112	26
	22	115	157	279	153	144	283	72	180		24	9	90	135	123	173	212	20	105
	27	0	0	0	0	0	0	0	0		52	0	0	0	0	0	0	0	0
	0	183	278	289	158	80	294	6	199		1	95	100	222	175	144	101	4	73
	6	22	257	21	119	144	73	27	22		7	177	215	308	49	144	297	49	149
	10	28	1	293	113	169	330	163	23		22	172	258	86	177	166	279	125	175
	11	67	351	13	21	90	99	50	100		25	61	256	162	128	19	222	194	108
	13	244	92	232	63	59	172	48	92		53	0	0	0	0	0	0	0	0
	17	11	253	302	51	177	150	24	207		0	221	102	210	192	0	351	6	103
7	18	157	18	138	136	151	284	38	52	32	12	112	201	22	209	211	265	126	110
	20	211	225	235	116	108	305	91	13		14	199	175	271	58	36	338	63	151
	28	0	0	0	0	0	0	0	0		24	121	287	217	30	162	83	20	211
	0	220	9	12	17	166	3	145	77		54	0	0	0	0	0	0	0	0
	1	44	62	88	76	189	103	88	146		1	2	323	170	114	0	56	10	199
	4	159	316	207	104	154	224	112	209		2	187	8	20	49	0	304	30	132
	7	31	333	50	100	184	297	153	32		11	41	361	140	161	76	141	6	172
	8	167	290	25	150	104	215	159	166		21	211	105	33	137	18	101	92	65
	14	104	114	76	158	164	39	76	18		55	0	0	0	0	0	0	0	0
	29	0	0	0	0	0	0	0	0		0	127	230	187	82	197	60	4	181
8	0	112	307	295	33	54	348	172	181	33	7	167	148	296	188	0	320	153	237
	1	4	179	133	95	0	75	2	105		15	164	202	5	68	108	112	197	142
	3	7	165	130	4	262	22	131	141		17	159	312	44	150	0	54	155	180
	12	211	18	231	217	41	312	141	223		56	0	0	0	0	0	0	0	0
	16	102	39	296	204	98	224	96	177		1	161	320	207	192	199	100	4	231
	19	164	224	110	39	46	17	99	145		6	197	335	158	173	278	210	45	174
	21	109	388	289	58	15	59	101	199		12	207	2	56	26	0	195	168	145
	22	241	67	245	44	230	314	35	153		22	103	266	285	187	205	268	185	100
	24	90	170	154	201	54	244	116	38		57	0	0	0	0	0	0	0	0
	30	0	0	0	0	0	0	0	0		0	37	210	259	222	216	135	6	11
9	0	103	366	189	9	162	156	6	169	34	14	105	313	179	157	16	15	200	207
	1	182	232	244	37	159	88	10	12		15	51	297	178	0	0	35	177	42
	10	109	321	36	213	93	293	145	206		18	120	21	160	6	0	188	43	100
	11	21	133	286	105	134	111	53	221		58	0	0	0	0	0	0	0	0
	13	142	57	151	89	45	92	201	17		1	198	269	298	81	72	319	82	59
	17	14	303	267	185	132	152	4	212		13	220	82	15	196	144	236	2	204
	18	61	63	135	109	76	23	164	92		23	122	115	115	138	0	85	135	161
	20	216	82	209	218	209	337	173	205		59	0	0	0	0	0	0	0	0
	31	0	0	0	0	0	0	0	0		0	167	185	151	123	190	164	91	121
	1	98	101	14	82	178	175	126	118		9	151	177	179	90	0	196	84	90
10	2	149	339	80	165	1	253	77	151	35	10	157	289	84	73	0	209	198	26
	4	167	274	211	174	28	27	156	70		12	163	214	181	10	0	246	100	140
	7	180	111	75	19	267	231	16	230		60	0	0	0	0	0	0	0	0
	8	49	383	161	194	234	49	12	115		1	173	258	102	12	153	236	4	115
	14	58	354	311	103	201	267	70	84		3	139	93	77	77	0	264	28	188
	32	0	0	0	0	0	0	0	0		7	149	346	192	49	165	37	109	168
	0	77	48	16	52	55	25	184	45		19	0	297	208	114	117	272	188	52
	1	41	102	147	11	23	322	164	115		61	0	0	0	0	0	0	0	0
	12	83	8	290	2	274	200	123	134		0	157	175	32	67	216	304	10	4
	16	182	47	289	35	181	351	16	1		8	137	37	80	45	144	237	84	103
11	21	78	188	177	32	273	166	104	152	36	17	149	312	197	96	2	135	12	30
	22	252	334	43	84	39	338	109	165		62	0	0	0	0	0	0	0	0
	23	22	115	280	201	26	162	124	107		1	167	52	154	23	0	123	2	53
	33	0	0	0	0	0	0	0	0		3	173	314	47	215	0	77	75	189
	0	160	77	229	142	225	123	6	186		9	139	139	124	60	0	25	142	215
	1	42	186	235	175	162	217	20	215		18	151	288	207	167	183	272	128	24
	10	21	174	169	136	244	142	203	124		63	0	0	0	0	0	0	0	0
	11	32	232	48	3	151	110	153	180		0	149	113	226	114	27	288	163	222
	13	234	50	105	28	238	176	104	98		4	157	14	65	91	0	83	10	170
	18	7	74	52	182	243	76	207	80		24	137	218	126	78	35	17	162	71
12	34	0	0	0	0	0	0	0	0	37	64	0	0	0	0	0	0	0	0
	0	177	313	39	81	231	311	52	220		1	151	113	228	206	52	210	1	22
	3	248	177	302	56	0	251	147	185		16	163	132	69	22	243	3	163	127
	7	151	266	303	72	216	265	1	154		18	173	114	176	134	0	53	99	49
	20	185	115	160	217	47	94	16	178		25	139	168	102	161	270	167	98	125
	23	62	370	37	78	36	81	46	150		65	0	0	0	0	0	0	0	0
	35	0	0	0	0	0	0	0	0		0	139	80	234	84	18	79	4	191
	0	206	142	78	14	0	22	1	124		7	157	78	227	4	0	244	6	211
	12	55	248	299	175	186	322	202	144		9	163	163	259	9	0	293	142	187
	15	206	137	54	211	253	277	118	182		22	173	274	260	12	57	272	3	148
13	16	127	89	61	191	16	156	130	95	38	66	0	0	0	0	0	0	0	0
	17	16	347	179	51	0	66	1	72		1	149	135	101	184	168	82	181	177
	21	229	12	258	43	79	78	2	76		6	151	149	228	121	0	67	45	114
	36	0	0	0	0	0	0	0	0		10	167	15	126	29	144	235	153	93
	0	40	241	229	90	170	176	173	39		67	0	0	0	0	0	0	0	0
	10	73	229	23	130	90	16	88	199		13	120	280	105	210	252	95	112	26
	24	9	90	135	123	173	212	20	105		24	9	90	135	123	173	212	20	105
	52	0	0	0	0	0	0	0	0		52	0	0	0	0	0	0	0	0
	1	95	100	222	175	144	101	4	73		1	95	100	222	175	144	101	4	73
	7	177	215	308	49	144	297	49	149		7	177	215	308	49	144	297	49	149
14	22	172	258	86	177	166	279	125	175	39	22	172	258	86	177	166	279	125	175
	25	61	256	162	128	19	222	194	108		25	61	256	162	128	19	222	194	108
	53	0	0	0	0	0	0	0	0		53	0	0	0	0	0	0	0	0
	0	221	102	210	192	0	351	6	103		0	221	102	210	192	0	351	6	103
	12	112	201	22	209	211	265	126	110		12	112	201	22	209	211	265	126	110
	14	199	175	271	58	36	338	63	151		14	199	175	271	58	36	338	63	151
	24	121	287	217	30	162	83	20	211		24	121	287	217	30	162	83	20	211
	54	0	0	0	0	0	0	0	0		54	0	0	0	0	0	0	0	0
	1	2	323	170	114	0	56	10	199		1	2	323	170	114	0	56	10	199
	2	187	8	20	49	0	304	30	132		2	187	8	20	49	0	304	30	132
15	11	41	361	140	161	76	141	6	172	40	21	211	105	33	137	18	101	92	

- **Tanner Graph Operations:**
 - LDPC decoding operates on a Tanner graph, a bipartite graph with **variable nodes** (representing codeword bits) and **check nodes** (representing parity-check equations from the H matrix).
 - In each iteration, messages (usually log-likelihood ratios, LLRs) are passed between variable nodes and check nodes along edges of the graph.
 - For a codeword of length n , with m check nodes and an average degree of connectivity (number of 1s per row/column in number of messages passed per iteration is proportional to the number of edges, which can be large (e.g., $3m$ to $5m$)).
- **Multiple Iterations:**
 - The decoder typically runs for 10–50 iterations to converge to a solution (i.e., until $H \times c^T = 0$ or a maximum iteration limit reached).
 - Each iteration involves:
 - **Variable-to-Check Messages:** Each variable node sends updated LLRs to connected check nodes.
 - **Check-to-Variable Messages:** Each check node computes new LLRs based on the incoming messages and sends back to variable nodes.
 - These computations involve additions, comparisons, and sometimes nonlinear operations (e.g., in the Sum-Product algorithm, computing hyperbolic tangents or their approximations).
 - For a codeword of length $n = 8448$ (a typical maximum in 5G NR Base Graph 1), with 50 iterations, the total number of operations can easily reach millions per codeword.
- **In 3GPP:**
 - 5G NR uses large block lengths (up to 8448 bits for BG1) and high code rates, leading to large Tanner graphs.
 - The high throughput requirements (e.g., tens of Gbps) mean the decoder must process many codewords per second, amplifying the computational load.

Large Parity-Check Matrix

The size and structure of the parity-check matrix H contribute to the computational intensity:

- **Matrix Dimensions:**
 - In 3GPP 5G NR, the H matrix is derived from a base graph (e.g., 46×68 for BG1) and expanded by a lifting factor Z (up to 384). This results in an H matrix with dimensions like $(46Z) \times (68Z)$, or up to 17664×26112 for $Z = 384$.
 - Even though H is sparse (low-density), the sheer size means there are still thousands of 1s, each corresponding to an edge in the Tanner graph.
- **Message Updates:**
 - Each 1 in H represents an edge in the Tanner graph, requiring a message to be computed and passed in each iteration.
 - For example, if a check node is connected to 10 variable nodes (a typical degree in 5G NR), it must process 10 incoming messages to compute 10 outgoing messages, often involving complex operations like minimum finding (in Min-Sum) or tanh computations (in Sum-Product).
- **In 3GPP:**
 - The quasi-cyclic structure of H (using circulant submatrices) helps with parallelization, but the large size still demands significant computation, especially for high Z values used in high-throughput scenarios.

Floating-Point or Fixed-Point Arithmetic

- **LLR Computations:**
 - During decoding, messages are typically LLRs, which are real numbers representing the likelihood of a bit being 0 or 1.
 - In the Sum-Product algorithm, check node updates involve nonlinear operations like $\tanh$ and $\operatorname{arctanh}$, which are computationally expensive.
 - Even in the Min-Sum algorithm (a simpler approximation used in 5G NR), check node updates require finding the minimum of incoming LLR magnitudes and computing signs, which still involves comparisons and multiplications.
- **Precision Requirements:**
 - To achieve good performance, LLRs are often represented with high precision (e.g., 6–8 bits in fixed-point implementation), increasing the computational cost of each operation.
 - In hardware implementations (e.g., ASICs for 5G base stations), fixed-point arithmetic is used to reduce complexity, but it requires careful design to avoid performance degradation.
- **In 3GPP:**
 - The high data rates of 5G NR require decoders to process LLRs at extremely high speeds, often in parallel, which demand significant computational resources.

Encoding Complexity (Though Less Intensive Than Decoding)

While encoding is generally less compute-intensive than decoding in LDPC, it still contributes to the overall computational load:

- **Parity Bit Computation:**
 - Encoding involves solving for parity bits such that $H \times c^T = 0$. In 3GPP, the H matrix is structured to make this efficient (e.g., using a lower-triangular form for the parity part), but it still requires matrix operations.
 - For a codeword of length n with k information bits, the number of parity bits is $n - k$, and solving for them involves operations proportional to $(n - k)^2$ in a naive implementation.
- **In 3GPP:**
 - The quasi-cyclic structure of H allows encoding to be performed using shift registers, reducing complexity to roughly linear in n .
 - However, for large block lengths (e.g., 8448 bits), encoding still requires significant computation, especially when processing many codewords per second.

High Throughput Requirements in 5G NR

- **Real-Time Constraints:**
 - 5G NR targets data rates of tens of Gbps, meaning the decoder must process millions of bits per second.
 - For a codeword of 8448 bits, at 10 Gbps, the decoder must handle over 1 million codewords per second, with each codeword requiring multiple iterations of message passing.
- **Parallel Processing:**
 - To meet these throughput demands, LDPC decoders in 5G NR are highly parallelized, processing multiple nodes or layers of the Tanner graph simultaneously.
 - While parallelism reduces latency, it increases the computational load, as more operations must be performed concurrently, requiring more hardware resources (e.g., in ASICs or FPGAs).
- **HARQ Support:**
 - 5G NR uses Hybrid Automatic Repeat Request (HARQ), where retransmissions send additional parity bits. The decoder must combine these with previous transmissions, increasing the computational complexity of soft combining and decoding.

Memory and Data Movement

- **Memory Access:**
 - LDPC decoding requires storing and accessing large amounts of data, including:
 - The H matrix (or its compact representation in 3GPP).
 - LLRs for each variable node.
 - Messages passed along edges in each iteration.
 - For a large H matrix (e.g., 17664×26112), even a sparse representation requires significant memory, and frequent memory accesses (to fetch and update messages) add to the computational burden.

- **In 3GPP:**
 - The quasi-cyclic structure reduces memory requirements by storing only the base graph and shift values, but the high throughput still demands fast memory access, often a bottleneck in hardware implementations.

Mitigations in 3GPP

While LDPC is compute-intensive, 3GPP's 5G NR design includes several optimizations to manage this complexity:

- **Quasi-Cyclic Structure:**
 - The H matrix is constructed from circulant submatrices, enabling parallel processing and reducing memory usage.
 - This allows hardware to process entire circulants (of size Z) in parallel, improving throughput.
- **Layered Decoding:**
 - Instead of updating all check nodes simultaneously, 5G NR often uses layered decoding, where check nodes are grouped into layers (based on the base graph structure). This reduces memory conflicts and improves convergence speed.
- **Min-Sum Algorithm:**
 - 5G NR typically uses the Min-Sum algorithm instead of the more complex Sum-Product algorithm, trading off some performance for lower computational complexity.
- **Hardware Acceleration:**
 - 5G base stations and devices use dedicated hardware (e.g., ASICs, FPGAs) to parallelize LDPC decoding, meeting the real-time requirements.

Comparison to Other Codes

- Compared to **Turbo codes** (used in LTE), LDPC codes in 5G NR are more compute-intensive per iteration due to the larger block sizes and more complex message-passing. However, LDPC converges faster (fewer iterations) and has better performance at high code rates, making it a net win for 5G.
- Compared to **Polar codes** (used for 5G control channels), LDPC is more compute-intensive because Polar codes use a simpler successive cancellation decoding, though they are less effective for large block lengths.

Reference

- [1] [3GPP TSG RAN WG1 #86 - R1-166414 Discussion on LDPC codes for NR](#)
- [2] [3GPP TSG RAN WG1 #86 R1-166769 Discussion on Length-Compatible Quasi-Cyclic LDPC Codes](#)
- [3] [3GPP TSG RAN WG1 #86 R1-166770 Discussion on Rate-Compatible Quasi-Cyclic LDPC Codes](#)
- [4] [3GPP TSG RAN WG1 #86 R1-166929 LDPC Code Design for NR](#)
- [5] [3GPP TSG-RAN WG1 #86 R1-167532 Discussion on LDPC coding scheme of code structure, granularity and HARQ-IR](#)
- [6] [3GPP TSG RAN WG1 #86 R1-167889 Design of Flexible LDPC Codes](#)

